

TABLE CONTENTS

S.NO	TOPICS	P.NO
1	Abstract	1
2	Introduction	2-3
3	Objective	4-5
4	Dataset Description	6-8
5	Methodology	9-12
6	Data Preprocessing	13- 16
7	Machine Learning Algorithms	17-19
8	Evaluation Metrics	20-24
9	Results and Comparison	25-26
10	Confusion Matrix Analysis	27-29
11	Best Performing Algorithm	29-31
12	Sample code and Outputs	32-39
13	Conclusion	40
14	References	41

ABSTRACT

Heart disease is one of the major health-related problems affecting people across the world. Early identification of heart disease can help in providing timely medical support and reducing the risk of serious complications. With the development of Artificial Intelligence and Machine Learning, it has become possible to analyze medical datasets and build predictive models that can assist in identifying whether a person is likely to have heart disease. Machine Learning algorithms can learn patterns from previously collected patient data and use those patterns to make predictions for new patients. This project focuses on developing a machine learning-based system for predicting the presence of heart disease using a structured medical dataset. The main purpose of this individual task is to select a suitable dataset, apply different machine learning algorithms, evaluate their performance, compare their results, and identify the best-performing algorithm. The selected dataset contains different medical and demographic attributes related to patients. These attributes include Age, Sex, Chest Pain Type, Resting Blood Pressure, Cholesterol level, Fasting Blood Sugar, Resting Electrocardiographic Results, Maximum Heart Rate Achieved, Exercise-Induced Angina, Oldpeak, ST Slope, and other relevant features. The target variable represents whether the patient has heart disease or not. The dataset therefore provides useful information for developing a binary classification model. Before applying machine learning algorithms, the dataset is carefully examined and preprocessed. Dataset exploration is performed to understand the number of records, features, data types, missing values, duplicate records, and distribution of the target variable. Data preprocessing is an important stage because machine learning algorithms require data in an appropriate numerical and consistent format. Categorical variables are converted into numerical representations using suitable encoding techniques. Numerical features are examined for their ranges and distributions, and feature scaling is applied when required by the selected algorithms. The data is then divided into training and testing datasets so that the models can be trained on one portion of the data and evaluated on unseen data. In this project, multiple machine learning classification algorithms are applied to the heart disease dataset. Depending on the selected implementation, algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, or K-Nearest Neighbors can be used for comparison. Each algorithm has a different method of learning patterns from the data. Logistic Regression is a statistical classification algorithm that estimates the probability of a class. Decision Tree uses a tree-like structure to make decisions based on feature values. Random Forest combines multiple decision trees to improve prediction performance and reduce the risk of overfitting. Other classification algorithms can also be included depending on the requirements of the individual task.

The dataset is divided into training and testing portions, commonly using an 80:20 ratio. The training data is used to teach the machine learning models, while the testing data is used to measure how well the trained models perform on previously unseen observations. Using a separate testing dataset provides a more realistic evaluation of the predictive ability of the models. A fixed random state can be used during splitting so that the experiment produces reproducible results. Stratified splitting may also be used to maintain a similar distribution of the target classes in both training and testing datasets.

To evaluate the performance of the machine learning algorithms, several classification evaluation metrics are considered. Accuracy measures the overall percentage of correctly classified instances. Precision measures how many of the patients predicted as having heart disease actually belong to the positive class. Recall, also known as sensitivity, measures how effectively the model identifies patients who actually have heart disease. F1-score provides a balance between precision and recall. In addition to these numerical metrics, a confusion matrix is used to provide a detailed view of correct and incorrect predictions. The confusion matrix contains True Positives, True Negatives, False Positives, and False Negatives. The comparison of different algorithms is an important part of this project. The performance results obtained from each model are organized in a comparison table, making it easier to identify differences between the algorithms. The model with the highest overall performance is considered the best-performing algorithm for the selected dataset, while considering not only accuracy but also other important metrics such as precision, recall, and F1-score. In a medical prediction problem, recall can be particularly important because failing to identify a patient who actually has heart disease may have serious consequences. Therefore, model selection should not depend only on accuracy.

INTRODUCTION

Heart disease is one of the most important health problems affecting people around the world. It includes different conditions that affect the heart and blood vessels and can lead to serious health complications if they are not identified and managed at an early stage. Factors such as age, gender, blood pressure, cholesterol level, chest pain, heart rate, lifestyle, and other medical conditions can be associated with the risk of developing heart disease. Because of this, early identification of individuals who may be at higher risk is an important part of healthcare. Traditional medical diagnosis is mainly based on a patient's medical history, physical examination, laboratory tests, and clinical experience. However, the increasing availability of healthcare data has created opportunities to use computational techniques to support medical analysis. Machine Learning is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every individual situation. Machine Learning has applications in many areas such as education, finance, transportation, cybersecurity, agriculture, and healthcare. In healthcare, machine learning techniques can be used to analyze patient information, identify patterns, classify medical conditions, and develop predictive models. Heart disease prediction is one such application where machine learning algorithms can be trained using information collected from patients and used to predict whether a particular patient belongs to a heart-disease or non-heart-disease category. The main idea behind a machine learning-based heart disease prediction system is to learn from historical patient records. Each record contains different characteristics or features of a patient, while the target variable indicates the presence or absence of heart disease. By providing these examples to a machine learning algorithm, the algorithm attempts to identify relationships and patterns between the input features and the target output. After learning from the training data, the model can be tested using previously unseen patient records. The predictions produced by the model can then be compared with the actual outcomes to determine how well the algorithm performs. The dataset used in this project contains medical and demographic information related to patients. Some of the important attributes include Age, Sex, Chest Pain Type, Resting Blood Pressure, Cholesterol, Fasting Blood Sugar, Resting Electrocardiographic Results, Maximum Heart Rate Achieved, Exercise-Induced Angina, Oldpeak, ST Slope, and other related variables. These features provide information about different characteristics of the patients and can be used as input variables for machine learning models. The target variable represents the presence or absence of heart disease. Since the target contains two possible classes, the problem can be treated as a binary classification problem. Data quality is an important factor in developing a reliable machine learning model. Raw datasets may contain missing values, duplicate records, categorical values, inconsistent data types, or numerical features with different ranges. Therefore, the dataset needs to be examined and prepared before applying machine learning algorithms. Data preprocessing helps convert the raw dataset into a suitable format for model training. The preprocessing stage may include checking missing values, removing duplicate records when appropriate, converting categorical variables into numerical values, checking unusual values, and scaling numerical features when required. Dataset exploration is another important step in the project. Before training the models, the dataset is examined to understand its structure and characteristics. Basic Python libraries such as Pandas can be used to display the first few records, identify the number of rows and columns, check data types, and calculate statistical information. The distribution of the target variable can also be examined to understand how many records belong to each class. This information helps in understanding whether the dataset is reasonably balanced or whether one class occurs more frequently than the other. Data visualization can also provide useful information about the dataset. Graphs and charts can be used to understand the distribution of important features and the relationship between different variables. For example, histograms can show the distribution of age, cholesterol, or resting blood pressure. Count plots can be used to display the number of patients in different categories. Correlation analysis can help identify relationships between numerical variables. Visualization makes the data easier to understand and can help identify patterns or unusual observations before model training. After preprocessing and exploration, the dataset is divided into training and testing sets. The training dataset is used to train the machine learning models, while the testing dataset is used to evaluate their performance. An 80:20 train-test split can be used, where approximately 80 percent of the data is used for training and 20 percent is reserved for testing. A fixed random state can be used to make the experiment reproducible. Stratified

splitting can also be applied for classification problems so that the proportion of the target classes remains similar in the training and testing datasets. Different machine learning algorithms can produce different results when applied to the same dataset. Therefore, applying more than one algorithm and comparing their performance is an important part of this individual task. Classification algorithms learn different types of decision boundaries and relationships between input features and target classes. In this project, two or three suitable machine learning classification algorithms are applied to the heart disease dataset according to the task requirements. The models are trained using the same prepared dataset so that their performance can be compared fairly. One possible algorithm for this problem is Logistic Regression. Logistic Regression is a commonly used classification algorithm that predicts the probability of an observation belonging to a particular class. It is relatively simple, efficient, and useful for binary classification problems. Another possible algorithm is Decision Tree, which uses a tree-like structure to make decisions based on feature values. A Random Forest can also be used, which combines multiple decision trees and can provide strong classification performance. The exact algorithms used in the final analysis depend on the selected models in the Jupyter Notebook. Model evaluation is an essential stage because simply training a model does not indicate whether it performs well. Several evaluation metrics can be used to measure the quality of the predictions. Accuracy represents the proportion of total predictions that are correct. Precision indicates how many of the observations predicted as positive are actually positive. Recall, or sensitivity, measures how effectively the model identifies actual positive cases. The F1-score combines precision and recall into a single measure. These metrics provide different perspectives on model performance and are useful for comparing classification algorithms. A confusion matrix is also used to analyze the predictions in greater detail. It contains four important outcomes: True Positive, True Negative, False Positive, and False Negative. A True Positive occurs when the model correctly predicts that a patient has heart disease. A True Negative occurs when the model correctly predicts that a patient does not have heart disease. A False Positive occurs when the model predicts heart disease for a patient who does not have it, while a False Negative occurs when the model fails to identify heart disease in a patient who actually has it. Examining these values helps in understanding the strengths and weaknesses of each model. The purpose of comparing multiple algorithms is to determine which model provides the most suitable performance for the selected dataset. The model with the highest accuracy is not automatically the best model in every healthcare-related problem. Precision, recall, F1-score, and the confusion matrix should also be considered. In particular, recall can be an important metric in heart disease prediction because identifying patients who actually have heart disease is an important objective. Therefore, the final model should be selected based on an overall evaluation of the results rather than relying on a single metric. This project is implemented using Python in a Jupyter Notebook environment. Python provides several libraries that simplify the process of data analysis and machine learning. Pandas and NumPy can be used for data manipulation and numerical operations. Matplotlib and Seaborn can be used for creating graphs and visualizations. Scikit-learn provides various machine learning algorithms, preprocessing methods, data-splitting functions, and evaluation metrics. Jupyter Notebook allows all the steps of the project to be organized in a single document, including explanations, Python code, outputs, graphs, and model results. The project also provides practical knowledge about the complete machine learning workflow. Instead of only studying machine learning algorithms theoretically, the project demonstrates how these algorithms can be applied to a real-world healthcare-related classification problem. The workflow begins with understanding the problem and selecting the dataset, followed by dataset exploration, preprocessing, splitting the data, training the models, generating predictions, evaluating the models, comparing their performance, and identifying the best-performing algorithm. It is important to understand that a machine learning prediction system is not a replacement for professional medical diagnosis. The predictions generated by a model depend on the quality, size, and characteristics of the dataset used for training. Real-world medical decisions require clinical knowledge, medical examinations, laboratory tests, and professional judgment. Therefore, this project is primarily an academic demonstration of how machine learning techniques can be applied to healthcare data and should not be considered a standalone medical diagnostic system. In conclusion, this project investigates the use of machine learning for heart disease prediction using patient-related features. By preprocessing the dataset, applying multiple classification algorithms, evaluating their predictions using appropriate metrics, and comparing their results, the project demonstrates the practical use of Artificial Intelligence and Machine Learning in a healthcare-related application.

OBJECTIVE

The main objective of this project is to develop a machine learning-based system for predicting the presence of heart disease using patient-related medical information. The project aims to demonstrate how machine learning classification techniques can be applied to a healthcare dataset to identify patterns in patient data and make predictions. The project also focuses on understanding the complete machine learning process, starting from dataset selection and data preprocessing and continuing through model training, evaluation, comparison, and identification of the best-performing algorithm. The first objective of the project is to understand the heart disease dataset and the information contained within it. The dataset consists of different medical and demographic attributes such as age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting electrocardiographic results, maximum heart rate achieved, exercise-induced angina, oldpeak, ST slope, and other relevant features. Understanding these attributes is important because each feature may provide useful information for predicting the target variable. The target variable indicates whether a patient has heart disease or does not have heart disease. The second objective is to perform systematic dataset exploration. Before applying any machine learning algorithm, it is necessary to understand the structure and quality of the available data. Therefore, the project aims to identify the number of records and features, examine the data types, check for missing values, identify duplicate records, and study the distribution of the target classes. Statistical summaries and visualizations are also used to understand the characteristics of the dataset. This step helps ensure that the data is properly understood before it is provided to the machine learning models. The third objective is to perform appropriate data preprocessing. Machine learning algorithms generally require data to be in a suitable numerical format. The dataset may contain categorical variables that cannot be directly processed by certain algorithms. Therefore, categorical features are converted into appropriate numerical representations using encoding techniques. Numerical features are also examined for their ranges and distributions. Where necessary, feature scaling is performed so that features with larger numerical values do not have an unnecessary influence on algorithms that are sensitive to feature magnitude. Another important objective is to divide the prepared dataset into training and testing sets. The training data is used by the machine learning algorithms to learn patterns from existing patient records, while the testing data is used to determine how well the trained models perform on unseen records. An 80:20 train-test split can be used for this project, where approximately 80 percent of the data is used for training and 20 percent is used for testing. A fixed random state can be used to ensure that the experiment can be reproduced consistently. The project also aims to apply two or three suitable machine learning classification algorithms to the selected heart disease dataset. The algorithms are trained using the prepared training data and then used to predict the target values of the testing data. Applying multiple algorithms is important because different algorithms learn patterns in different ways and may produce different levels of performance on the same dataset. Another objective is to understand the working and practical application of the selected machine learning algorithms. For example, Logistic Regression can be used to estimate the probability of a patient belonging to a particular class. Decision Tree can classify observations through a sequence of decision rules, while Random Forest can combine multiple decision trees to produce predictions. Studying and applying different algorithms provides a better understanding of how machine learning methods can be used to solve classification problems. An important objective of this project is to evaluate the performance of each machine learning model using suitable classification evaluation metrics. Accuracy, precision, recall, and F1-score can be calculated for each model. Accuracy provides an overall measure of correct predictions, while precision measures the correctness of positive predictions. Recall measures the ability of the model to correctly identify positive cases, and F1-score provides a balance between precision and recall. Using multiple evaluation metrics allows the models to be compared more effectively. The project also aims to analyze the confusion matrix generated by each classification model. A confusion matrix provides detailed information about the predictions made by a model. It shows True Positives, True Negatives, False Positives, and False Negatives. Studying these values helps in understanding the types of errors made by each algorithm. In a heart disease prediction problem, this analysis is particularly useful because incorrect predictions can have different implications depending on whether the error is a false positive or a false negative. Another major objective is to

compare the performance of the selected machine learning algorithms. The evaluation results of all models are organized in a comparison table so that their performance can be clearly observed. The comparison is based on metrics such as accuracy, precision, recall, and F1-score. Graphical representations can also be used to make the comparison easier to understand. This comparison helps determine which algorithm provides the most suitable results for the selected dataset. The project aims to identify the best-performing algorithm among the models used. The best model is selected based on an overall evaluation of its performance rather than considering only one metric. Although accuracy is an important measure, precision, recall, F1-score, and confusion matrix results are also considered. For a healthcare-related classification problem, recall can be particularly important because correctly identifying positive heart disease cases is significant. Therefore, the final selection should consider the specific requirements of the problem as well as the observed results. Another objective is to understand the importance of visualization in machine learning projects. Graphs such as count plots, histograms, correlation heatmaps, and confusion matrix visualizations can be used to present information clearly. Visualization makes it easier to identify patterns, understand feature distributions, examine relationships between variables, and communicate model performance. It also makes the project report more informative and understandable. The project further aims to gain practical experience with Python-based machine learning tools. Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn are used to perform different stages of the project. Pandas can be used to load and manipulate the dataset, NumPy can support numerical operations, Matplotlib and Seaborn can be used for visualization, and Scikit-learn can be used for preprocessing, model training, prediction, and evaluation. Working with these libraries provides practical knowledge that is useful for future machine learning projects. The project also aims to understand the importance of reproducibility in machine learning experiments. By using fixed parameters such as a random state during dataset splitting and model training where appropriate, similar results can be obtained when the notebook is executed again. Maintaining a clear sequence of steps, documenting the preprocessing methods, and recording evaluation results makes the experiment easier to understand and reproduce. Finally, the overall objective of this project is to demonstrate the practical application of machine learning to a real-world healthcare-related classification problem. The project combines data analysis, preprocessing, machine learning, evaluation, visualization, and comparative analysis into one complete workflow. It provides an opportunity to understand how raw patient information can be transformed into useful predictive information through machine learning techniques.

Therefore, the major objectives of the project can be summarized as follows:

1. To understand and analyze the selected heart disease dataset.
2. To identify the important features associated with the prediction task.
3. To examine the dataset for missing values, duplicate records, and other data-quality issues.
4. To preprocess categorical and numerical data appropriately.
5. To divide the dataset into training and testing datasets.
6. To apply two or three suitable machine learning classification algorithms.
7. To train the selected models using the training dataset.
8. To generate predictions using the testing dataset.
9. To evaluate each model using accuracy, precision, recall, and F1-score.
10. To analyze the confusion matrix of the selected models.
11. To compare the performance of the machine learning algorithms.
12. To identify the best-performing algorithm based on the evaluation results.
13. To visualize important dataset characteristics and model performance.
14. To gain practical experience with Python and machine learning libraries.
15. To understand the complete workflow of developing and evaluating a machine learning classification model.
16. To demonstrate the potential application of machine learning techniques to heart disease prediction.

Overall, this project is intended to provide both theoretical and practical understanding of machine learning classification. It shows how a healthcare-related dataset can be processed and analyzed and how different algorithms can be trained and evaluated to solve a prediction problem.

DATASET DESCRIPTION

The dataset used in this project is a heart disease dataset that contains medical and demographic information about patients. The purpose of using this dataset is to develop a machine learning classification model that can predict whether a patient has heart disease or not. Each row in the dataset represents an individual patient, while each column represents a particular characteristic or medical attribute of that patient. The target variable indicates the presence or absence of heart disease. Since the target variable has two possible outcomes, the problem is considered a binary classification problem. The dataset contains several features that provide information about the patient's age, gender, chest pain, blood pressure, cholesterol, blood sugar, electrocardiographic results, heart rate, exercise-related symptoms, and other cardiac-related measurements. These features are used as input variables for the machine learning algorithms. By learning from the relationship between these features and the target variable, the machine learning models attempt to predict the target class for new or previously unseen patient records.

Structure of the Dataset:

The dataset is stored in a CSV file named "heart.csv". CSV stands for Comma-Separated Values and is a commonly used format for storing tabular data. The file can be loaded into Python using the Pandas library. Once loaded into a Pandas DataFrame, the dataset can be examined using different functions to understand its structure. The dataset consists of rows and columns. Each row corresponds to one patient record, while the columns correspond to different features. The dataset contains both numerical and categorical variables. Numerical variables contain values that represent measurable quantities, while categorical variables contain different categories or labels.

The target column in the dataset is "HeartDisease". It represents whether heart disease is present in a particular patient. A value of "1" represents the presence of heart disease, while a value of "0" represents the absence of heart disease. Therefore, the machine learning algorithms are trained to learn the relationship between the input features and this target variable.

Features Present in the Dataset:

The major features included in the heart disease dataset are described below.

1. Age: The "Age" feature represents the age of the patient, generally measured in years. Age is an important demographic feature because the risk of several cardiovascular conditions can vary with age. The machine learning model can use age along with other features to identify patterns related to the target variable.

2. Sex: The "Sex" feature represents the sex of the patient. It is a categorical variable. In many versions of this dataset, the values are represented as "M" for male and "F" for female. Since machine learning algorithms generally require numerical input, categorical values such as these need to be encoded before model training.

3. ChestPainType: "ChestPainType" represents the type of chest pain experienced by the patient. Chest pain characteristics can provide useful information in the analysis of heart-related conditions. The feature contains different categories, commonly represented as "TA", "ATA", "NAP", and "ASY".

These categories represent different types of chest pain or absence of typical chest pain symptoms. Since this is a categorical feature, it needs to be converted into a numerical representation before being provided to machine learning algorithms.

4. RestingBP: "RestingBP" represents the patient's resting blood pressure. It is a numerical feature and is generally measured in millimeters of mercury (mmHg). Blood pressure is an important health measurement and can provide useful information when analyzing cardiovascular risk.

The values of this feature can vary significantly between patients. Therefore, it is important to examine the distribution of the feature during the data exploration stage and check for unusual or unrealistic values.

5. Cholesterol: The "Cholesterol" feature represents the patient's cholesterol measurement. Cholesterol is a numerical medical attribute and can be useful in studying cardiovascular health. Different patients can have significantly different cholesterol values.

During preprocessing and exploration, this feature can be analyzed using statistical summaries and visualizations. Its relationship with the target variable can also be investigated.

6. FastingBS: "FastingBS" represents fasting blood sugar. In this dataset, it is generally represented as a binary variable. A value of "1" indicates a fasting blood sugar level above the specified threshold, while "0" indicates a value at or below the threshold.

Because the feature contains two possible values, it can be directly used as a numerical binary feature in many machine learning workflows.

7. RestingECG: "RestingECG" represents the result of the resting electrocardiogram (ECG) examination. It is a categorical feature with different possible categories, commonly represented as "Normal", "ST", and "LVH".

ECG-related information can provide useful patterns for classification. However, because the feature contains categorical values, it must be encoded before being used by algorithms that require numerical input.

8. MaxHR: "MaxHR" represents the maximum heart rate achieved by the patient during the relevant examination or exercise test. It is a numerical feature.

Heart rate can vary according to factors such as age and physical condition. The machine learning model can use maximum heart rate together with the other patient features to identify patterns associated with the target variable.

9. ExerciseAngina: "ExerciseAngina" represents whether the patient experiences exercise-induced angina. It is a categorical variable, commonly represented by "Y" for Yes and "N" for No.

This feature is useful because exercise-related symptoms may contain information relevant to heart disease classification. Since it is categorical, it is converted into an appropriate numerical representation during preprocessing.

10. Oldpeak: "Oldpeak" is a numerical feature related to ST-segment depression observed during exercise compared with the resting condition. It is represented using numerical values and may contain decimal values.

The feature can have different values for different patients and can therefore contribute to the classification process. Numerical features such as Oldpeak may also benefit from scaling depending on the machine learning algorithms being used.

11. ST_Slope: "ST_Slope" represents the slope of the peak exercise ST segment. It is a categorical feature with categories such as "Up", "Flat", and "Down".

This feature needs to be converted into a numerical form before being used by most machine learning algorithms. One-hot encoding is one commonly used method for handling such categorical variables.

12. HeartDisease: "HeartDisease" is the target variable of the dataset. It represents whether the patient has heart disease.

The target generally contains two values:

- "0" No heart disease

- "1" Heart disease present

The objective of the machine learning models is to predict this target variable based on the other features in the dataset. Since there are two classes, this is a binary classification problem.

Numerical and Categorical Features

The dataset contains a combination of numerical and categorical features. Numerical features include variables such as Age, RestingBP, Cholesterol, MaxHR, and Oldpeak. These features contain numerical measurements and can be directly analyzed using statistical methods. Categorical features include Sex, ChestPainType, RestingECG, ExerciseAngina, and ST_Slope. These features contain labels or categories rather than continuous numerical measurements. Machine learning algorithms generally require numerical input, so categorical features need to be transformed into numerical representations during preprocessing. This combination of numerical and categorical variables makes preprocessing an important part of the project. Appropriate encoding and scaling methods must be selected according to the requirements of the algorithms being used.

Input and Target Variables

For the machine learning process, the dataset is divided into input variables and a target variable.

The input variables, commonly represented by "X", consist of the patient features used to make the prediction. These include age, sex, chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, Oldpeak, and ST slope.

The target variable, commonly represented by "y", is "HeartDisease". It contains the actual class associated with each patient record.

Conceptually, the machine learning task can be represented as:

Input Features → Machine Learning Model → Heart Disease Prediction

The model learns from the input features and their corresponding target values during the training stage. After training, it receives new input data and predicts whether the patient belongs to the heart disease or non-heart disease class.

Dataset Exploration

Before applying machine learning algorithms, the dataset must be explored carefully. The first few records can be displayed to understand the format of the data. The number of rows and columns can be checked to understand the size of the dataset. Data types can also be examined to identify which variables are numerical and which are categorical. Missing values should be checked because incomplete records can affect model training. Duplicate records should also be identified because repeated observations may influence the learning process. Statistical descriptions can be generated for numerical features to understand their minimum, maximum, average, and other statistical characteristics. The distribution of the target variable is also important. By counting the number of records belonging to class "0" and class "1", it is possible to understand whether the target classes are balanced or imbalanced. If one class is much more common than the other, evaluation metrics such as precision, recall, and F1-score become particularly important.

Importance of the Dataset

The selected dataset is suitable for this project because it contains multiple patient-related features and a clearly defined target variable. It provides an opportunity to apply classification algorithms and compare their performance. The dataset also demonstrates an important real-world machine learning challenge because it contains both numerical and categorical information. Therefore, the project requires several preprocessing steps before model training. Handling these different types of data provides practical experience in preparing real-world datasets for machine learning.

Role of the Dataset in the Project

The dataset forms the foundation of the entire project. The quality of the dataset directly influences the quality of the machine learning models. The dataset is first loaded into the Jupyter Notebook and examined carefully. After exploration, appropriate preprocessing techniques are applied. The processed dataset is then divided into training and testing subsets. The training subset is provided to the selected machine learning algorithms so that they can learn patterns from the data. The testing subset is kept separate and used to evaluate the models after training. Predictions generated by the models are compared with the actual values of "HeartDisease". Evaluation metrics such as accuracy, precision, recall, and F1-score are calculated. Confusion matrices are also generated to understand the types of correct and incorrect predictions made by each model.

METHODOLOGY

The methodology of this project describes the systematic procedure followed to develop a machine learning-based heart disease prediction system. The project follows a sequence of important steps, beginning with understanding the problem and dataset and continuing through data preprocessing, model development, prediction, evaluation, comparison, and identification of the best-performing algorithm. Following a structured methodology helps ensure that the machine learning experiment is organized, understandable, and reproducible. The main goal of the methodology is to use patient-related information to predict whether a patient has heart disease or not. Since the target variable contains two possible classes, the problem is treated as a binary classification problem. Different machine learning classification algorithms are trained using the same prepared dataset, and their performances are compared using suitable evaluation metrics.

Problem Definition

The first step of the methodology is to clearly define the problem. Heart disease prediction is selected as the machine learning problem because early identification of possible heart disease can be useful in healthcare-related analysis. The dataset contains different patient characteristics that can be used as input features, while the "HeartDisease" column represents the target output. The machine learning model is expected to learn the relationship between the input features and the target variable. After learning from the training data, the model is tested using unseen data to determine how accurately it can classify patients into the appropriate target category.

The problem can therefore be represented as:

Patient Information → Machine Learning Model → Heart Disease Prediction

The prediction belongs to one of two classes:

- 0: No heart disease
- 1: Heart disease present

Dataset Collection

The next step is to obtain the dataset required for the project. The selected dataset is stored in a CSV file named "heart.csv". The dataset contains patient-related medical and demographic information. The dataset is placed inside the project's dataset folder. In the Jupyter Notebook, the CSV file is loaded using the Pandas library. A DataFrame is created after reading the CSV file, which allows the dataset to be easily examined, manipulated, and processed.

For example, the dataset can be loaded using:

The loaded dataset is then stored in a variable such as "df", which is used throughout the analysis.

Dataset Exploration

After loading the dataset, the next step is dataset exploration. This step is performed to understand the structure and characteristics of the data before applying any machine learning algorithm. The first few records are displayed to understand how the values are organized. The number of rows and columns is checked to determine the size of the dataset. The names of the columns are examined to identify the available features. Data types are also checked because the dataset contains both numerical and categorical variables. Numerical variables can contain measurements such as age, blood pressure, cholesterol, and maximum heart rate, while categorical variables contain labels such as sex, chest pain type, and exercise-induced angina. The dataset is also examined for missing values and duplicate records.

Statistical information is calculated for numerical features to understand their distributions and ranges. The target variable is analyzed to determine the number of records belonging to each class.

Data Visualization

Visualization is an important part of the methodology because it provides a graphical understanding of the dataset. Different types of plots can be used to examine the distribution of features and identify patterns.

For example, count plots can be used to display the number of patients in each heart disease category. Histograms can show the distribution of numerical variables such as age, cholesterol, and maximum heart rate. Box plots can help identify unusual values or potential outliers.

A correlation heatmap can also be used to visualize relationships between numerical variables. Visualization allows the characteristics of the dataset to be understood more easily and can help identify possible issues before preprocessing and model training.

Data Preprocessing

Data preprocessing is one of the most important stages of the methodology. Machine learning algorithms require data to be in an appropriate format. Therefore, the raw dataset is prepared before model training.

First, missing values are checked. If missing values are present, an appropriate method can be used to handle them. Duplicate records are also identified and removed when necessary. The categorical features in the dataset need to be converted into numerical representations. Features such as "Sex", "ChestPainType", "RestingECG", "ExerciseAngina", and "ST_Slope" contain categorical values. Encoding techniques such as one-hot encoding can be used to transform these categories into numerical columns. Numerical features such as "Age", "RestingBP", "Cholesterol", "MaxHR", and "Oldpeak" are retained as numerical variables. Depending on the algorithms selected, numerical features may be scaled so that features with larger numerical ranges do not dominate features with smaller ranges. The objective of preprocessing is to create a clean and suitable dataset that can be effectively used by the selected machine learning algorithms.

Separation of Input and Target Variables

After preprocessing, the dataset is separated into input features and the target variable.

The input features are represented as "X". They contain all the variables that are used by the machine learning algorithms to make predictions. The "HeartDisease" column is excluded from "X" because it is the output that the model needs to predict.

The target variable is represented as "y" and contains the "HeartDisease" values.

Therefore: This separation is essential because the model should learn to predict the target using the input features rather than receiving the target as one of its input variables.

Train-Test Split

The prepared dataset is divided into training and testing datasets. The purpose of this step is to evaluate how well the model performs on data that it has not previously seen. An 80:20 split can be used for this project. Approximately 80 percent of the records are used for training and 20 percent are reserved for testing. The training dataset is used to teach the machine learning algorithms. The testing dataset is kept separate until after training and is then used to generate predictions. A fixed "random_state" can be used so that the same records are selected each time the notebook is executed. For a classification problem, stratification can also be applied to maintain a similar distribution of the target classes in both training and testing datasets.

Selection of Machine Learning Algorithms

After preparing and splitting the data, suitable machine learning algorithms are selected. The individual task requires applying two or three machine learning algorithms and comparing their performance. Classification algorithms are selected because the target variable has two classes. Suitable algorithms may include Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, or other appropriate classification techniques. Each algorithm approaches the classification problem differently. Therefore, applying multiple algorithms provides an opportunity to determine which method performs most effectively on the selected heart disease dataset.

Model Training

The selected algorithms are trained using the training dataset. During training, each algorithm analyzes the input features and corresponding target values and attempts to learn patterns that can be used for classification. For each selected model, the training data is provided as input along with the corresponding target values. The model then builds an internal representation of the relationship between the features and the target. The training process is performed separately for each algorithm. The same training and testing datasets should be used for all models so that the comparison is fair.

Prediction:

After training, each model is used to predict the target values for the testing dataset. The testing data contains patient features that were not used during model training. The trained model receives these features and produces a predicted class for each patient. The predicted values are then compared with the actual "HeartDisease" values in the testing dataset. Correct predictions and incorrect predictions are used to calculate the evaluation metrics.

Model Evaluation

After predictions are generated, the performance of each model is evaluated using suitable classification metrics.

The main metrics considered in the project are:

- Accuracy
- Precision
- Recall
- F1-score

Accuracy measures the percentage of total predictions that are correct. Precision measures how many of the patients predicted as having heart disease actually belong to the positive class. Recall measures how effectively the model identifies patients who actually have heart disease. F1-score provides a balance between precision and recall.

Using multiple metrics provides a more complete understanding of model performance.

Confusion Matrix

A confusion matrix is generated for each selected classification model. It provides a detailed representation of correct and incorrect predictions.

The four components of a confusion matrix are:

True Positive (TP): The model correctly predicts heart disease.

True Negative (TN): The model correctly predicts no heart disease.

False Positive (FP): The model predicts heart disease when the actual class is no heart disease.

False Negative (FN): The model predicts no heart disease when the actual class is heart disease.

Analyzing these four outcomes helps identify the types of errors made by each model.

Performance Comparison

After evaluating all models, their results are organized into a comparison table. The table contains the evaluation metrics obtained from each algorithm.

For example, the comparison can contain:

Algorithm	Accuracy	Precision	Recall	F1-Score
-----------	----------	-----------	--------	----------

Algorithm 1	Result	Result	Result	Result
-------------	--------	--------	--------	--------

Algorithm 2	Result	Result	Result	Result
-------------	--------	--------	--------	--------

Algorithm 3	Result	Result	Result	Result
-------------	--------	--------	--------	--------

The actual values are filled in after running the models in the Jupyter Notebook.

Graphs can also be used to compare the performance of the algorithms. A bar chart showing accuracy or multiple evaluation metrics can make the results easier to interpret.

Selection of the Best Model

The final stage of the methodology is selecting the best-performing algorithm. The decision is based on the evaluation results obtained from the testing dataset. The model with strong overall performance across the selected evaluation metrics can be considered the best-performing model for the dataset. Accuracy is considered, but precision, recall, F1-score, and confusion matrix results are also examined.

For a healthcare-related classification problem, recall can be particularly important because correctly identifying patients who actually have heart disease is an important consideration. Therefore, the best model should be selected based on the overall results rather than simply choosing the model with the highest accuracy.

Tools and Technologies Used

The project is implemented using Python in a Jupyter Notebook environment. Several Python libraries are used during different stages of the project. Pandas is used for loading, exploring, and manipulating the dataset. NumPy is used for numerical operations. Matplotlib and Seaborn are used for data visualization and graphical representation of results. Scikit-learn is used for data preprocessing, train-test splitting, machine learning algorithms, and evaluation metrics. Jupyter Notebook provides an interactive environment where Python code, explanations, tables, graphs, and outputs can be organized together. This makes it suitable for documenting the complete machine learning workflow.

Conclusion

This workflow provides a systematic approach to solving the heart disease classification problem. Each stage contributes to the final result and helps ensure that the machine learning models are developed and evaluated in an organized manner.

DATA PREPROCESSING

Data preprocessing is one of the most important stages in a machine learning project. Raw datasets are not always directly suitable for machine learning algorithms because they may contain categorical values, missing values, duplicate records, inconsistent formats, or numerical features with different scales. If such issues are not handled properly, they can negatively affect the performance of machine learning models. Therefore, before training the models, the heart disease dataset is carefully examined, cleaned, transformed, and prepared for the classification task. The main purpose of data preprocessing in this project is to convert the original heart disease dataset into a suitable format that can be effectively processed by the selected machine learning algorithms. The preprocessing procedure includes understanding the dataset, checking data quality, handling missing values, checking duplicate records, separating input and target variables, encoding categorical features, scaling numerical features when required, and finally dividing the prepared data into training and testing sets.

Loading the Dataset

The first step in preprocessing is loading the "heart.csv" file into the Python environment. The Pandas library is used to read the CSV file and store the information in a DataFrame. A DataFrame provides a convenient tabular structure for examining and manipulating the dataset. The dataset is loaded into the Jupyter Notebook using the appropriate file path. After loading the data, the first few rows are displayed to verify that the file has been loaded correctly. This initial inspection helps confirm that the expected columns and values are present. The dataset contains patient-related features such as Age, Sex, ChestPainType, RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope, and the target variable HeartDisease.

Understanding the Dataset Structure

After loading the dataset, its basic structure is examined. The number of rows and columns is identified to understand the size of the dataset. The column names are also checked to ensure that all expected features are available. The data types of the columns are examined because the dataset contains both numerical and categorical variables. Numerical columns contain values such as age, blood pressure, cholesterol, and heart rate. Categorical columns contain values such as male/female, chest pain categories, and exercise-induced angina categories.

Understanding the data types is important because different types of variables require different preprocessing techniques.

Checking for Missing Values

Missing values are an important data-quality issue that must be considered before model training. A missing value means that information for a particular feature is not available for a particular patient record. The dataset is checked for missing values in every column. If missing values are present, they need to be handled appropriately. Depending on the feature and the number of missing values, possible methods include removing affected records or replacing missing values using suitable statistical values such as the mean, median, or mode. If the dataset does not contain missing values, no imputation is required. The absence of missing values is useful because it allows the available records to be used without introducing estimated values.

Checking Duplicate Records

Duplicate records are also checked during preprocessing. A duplicate record occurs when the same patient information appears more than once in the dataset. Duplicate records can potentially influence the learning process because repeated observations may give certain patterns more importance than they

should have. Therefore, the dataset is examined for duplicate rows. If duplicate records are found and they do not represent separate legitimate observations, they can be removed. If no duplicates are present, the original dataset can be retained.

Identifying Numerical Features

The heart disease dataset contains several numerical features. These include:

- Age
- RestingBP
- Cholesterol
- MaxHR
- Oldpeak

These variables contain numerical measurements and can be analyzed statistically. Their minimum, maximum, mean, median, and other descriptive statistics can be examined.

Numerical features may have very different ranges. For example, Age may have values in a relatively small range, while Cholesterol and RestingBP may have larger numerical values. Some machine learning algorithms are sensitive to these differences in scale. Therefore, scaling may be required depending on the algorithms selected.

Identifying Categorical Features

The dataset also contains categorical features. These include:

- Sex
- ChestPainType
- RestingECG
- ExerciseAngina
- ST_Slope

Categorical variables represent categories rather than continuous numerical measurements. For example, "Sex" may contain "M" and "F", while "ExerciseAngina" may contain "Y" and "N".

Most machine learning algorithms cannot directly process text-based categories in their original form. Therefore, categorical features must be converted into numerical representations before model training.

Encoding Categorical Variables

Encoding is the process of converting categorical values into numerical values. This is necessary because machine learning algorithms generally require numerical input. For categorical variables with multiple categories, one-hot encoding can be used. One-hot encoding creates separate binary columns for the different categories. For example, a feature containing different chest pain categories can be transformed into separate indicator columns. For binary categorical features, such as "ExerciseAngina", values can be represented using numerical values such as 0 and 1.

Proper encoding ensures that the categorical information is retained while making the dataset compatible with machine learning algorithms.

Target Variable

The target variable in this project is "HeartDisease". It represents the class that the machine learning model needs to predict.

The target variable contains two classes:

- 0 — No heart disease
- 1 — Heart disease present

The target variable is separated from the input features before model training. The input features are represented as "X", while the target is represented as "y".

Conceptually:

X = Patient-related input features

y = HeartDisease

The model learns the relationship between "X" and "y" during the training process.

Feature Scaling

Feature scaling is another preprocessing technique that may be required for certain machine learning algorithms. Scaling converts numerical features to a comparable range.

For example, Age, RestingBP, Cholesterol, MaxHR, and Oldpeak can have different numerical ranges. If an algorithm is sensitive to feature magnitude, a feature with larger numerical values may have an unintended influence on the model.

Standardization is a commonly used scaling technique. It transforms numerical features so that they have a mean close to zero and a standard deviation close to one.

However, scaling should be performed carefully. The scaler should be fitted using the training data and then applied to the testing data. This prevents information from the test dataset from influencing the training process.

Checking Feature Distributions

The distribution of numerical features can be examined before model training. Histograms and box plots can be used to understand the distribution of variables such as Age, RestingBP, Cholesterol, MaxHR, and Oldpeak. Feature distributions may reveal unusual values or potential outliers. An outlier is an observation that is considerably different from the majority of observations. Outliers should not automatically be removed. They may represent legitimate patient measurements. Therefore, they should be examined carefully before deciding whether any treatment is necessary.

Correlation Analysis

Correlation analysis can be performed on numerical variables to understand relationships between features. A correlation matrix can be visualized using a heatmap. Correlation values indicate the strength and direction of linear relationships between numerical variables. A positive correlation indicates that two variables tend to increase together, while a negative correlation indicates that one variable tends to decrease as the other increases. Correlation analysis is useful for understanding the dataset and identifying potentially related features. However, correlation does not necessarily mean that one variable causes another.

Separating Input and Output

After the required preprocessing operations are completed, the input features and target variable are separated. The "HeartDisease" column is removed from the input feature dataset because it is the value that the model is expected to predict.

The remaining columns form "X", while "HeartDisease" forms "y".

For example, conceptually:

This separation is an essential step in supervised machine learning.

-

Train-Test Splitting

After preparing the features and target, the dataset is divided into training and testing sets. The training set is used to teach the machine learning algorithms. The testing set is used later to evaluate how well the trained models perform on unseen data.

An 80:20 ratio can be used, meaning approximately 80 percent of the records are used for training and 20 percent for testing. For classification problems, stratification can be used during the split. Stratification helps maintain a similar proportion of the two target classes in both the training and testing datasets.

A fixed random state, such as "random_state=42", can be used to make the split reproducible.

Avoiding Data Leakage

An important consideration during preprocessing is avoiding data leakage. Data leakage occurs when information from the testing dataset unintentionally influences the training process.

For example, when feature scaling is used, the scaler should be fitted only on the training data. The same fitted scaler is then used to transform the testing data. Similarly, any preprocessing operation that learns information from the data should be performed using the training portion rather than using the entire dataset before splitting. Avoiding data leakage helps ensure that the evaluation results provide a more realistic indication of how the model performs on unseen data.

Preparing Data for Machine Learning

After all required preprocessing steps are completed, the dataset is ready for machine learning. The input features are represented numerically, categorical variables have been encoded, numerical variables have been scaled when necessary, and the target variable is separated. The resulting training dataset is then provided to each selected machine learning algorithm. Each algorithm learns from the same prepared training data, and the same testing dataset is used to compare their performance.

Using the same prepared data for all algorithms makes the comparison more consistent and fair.

This sequence provides a systematic approach to preparing the dataset.

Importance of Data Preprocessing

Data preprocessing has a major influence on the success of a machine learning project. Even a powerful machine learning algorithm may produce poor results if the input data is not properly prepared. Correct preprocessing ensures that categorical variables can be understood by the algorithms, numerical features are appropriately represented, missing and duplicate records are handled, and the training and testing datasets are properly separated. Preprocessing also improves the reliability and consistency of the machine learning experiment. It allows the algorithms to focus on meaningful patterns in the dataset rather than being affected by avoidable data-quality problems.

Summary

In this project, data preprocessing is performed to transform the raw heart disease dataset into a suitable format for machine learning. The process begins by loading and inspecting the "heart.csv" dataset. The dataset is checked for missing values, duplicate records, data types, and unusual values. The numerical and categorical features are identified separately. Categorical features such as Sex, ChestPainType, RestingECG, ExerciseAngina, and ST_Slope are converted into numerical representations using appropriate encoding techniques. Numerical features such as Age, RestingBP, Cholesterol, MaxHR, and Oldpeak are examined and scaled when required by the selected algorithms. The target variable "HeartDisease" is separated from the input features. The prepared data is then divided into training and testing datasets, with an 80:20 split being suitable for the project. Stratification and a fixed random state can be used to maintain class distribution and reproducibility.

MACHINE LEARNING ALGORITHMS

Machine Learning algorithms are mathematical and computational methods that enable computers to learn patterns from data and make predictions or decisions. In this project, machine learning algorithms are used to predict whether a patient has heart disease or not based on different medical and demographic features. Since the target variable, "HeartDisease", contains two classes, the problem is treated as a binary classification problem. The selection of an appropriate machine learning algorithm is an important part of the project because different algorithms learn patterns in different ways. An algorithm that performs well on one dataset may not necessarily perform equally well on another dataset. Therefore, this project applies three classification algorithms and compares their performance using suitable evaluation metrics. The selected algorithms are Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. Using multiple algorithms provides a better understanding of the dataset and allows the performance of different machine learning approaches to be compared. All three models are trained using the same training data and evaluated using the same testing data. This makes it possible to perform a fair comparison between the algorithms.

Logistic Regression

Logistic Regression is a commonly used supervised machine learning algorithm for classification problems. Despite having the word "regression" in its name, Logistic Regression is primarily used for classification. It is particularly suitable for binary classification problems where the target variable has two possible classes. In this project, Logistic Regression is used to predict whether a patient belongs to the heart disease class or the non-heart disease class. The algorithm calculates a probability for the possible classes and uses a decision threshold to assign the final class.

For example, the model may produce a probability indicating how likely a patient is to belong to the heart disease class. Based on the classification threshold, the patient can then be assigned to class "0" or class "1".

The basic idea behind Logistic Regression is to calculate a weighted combination of the input features and transform the result using the logistic or sigmoid function. The sigmoid function converts the resulting value into a probability between 0 and 1.

The general form of the sigmoid function is:

$$\sigma(z) = 1 / (1 + e^{-z})$$

where "z" represents the weighted combination of the input features.

The model learns appropriate weights during the training process. These weights determine how the input features contribute to the prediction. For the heart disease dataset, features such as Age, RestingBP, Cholesterol, MaxHR, Oldpeak, and encoded categorical variables can contribute to the prediction. One important advantage of Logistic Regression is its simplicity. It is relatively easy to train, interpret, and implement. It also works efficiently with many classification datasets. Another advantage is that it can provide probability estimates, which can be useful when analyzing classification decisions. Logistic Regression is also often used as a baseline model because its performance can be compared with more complex algorithms. However, Logistic Regression has some limitations. It generally assumes a relatively simple relationship between the input variables and the target. If the relationship between features and the target is highly complex or nonlinear, a more flexible algorithm may perform better. In this project, Logistic Regression provides a useful baseline against which the performance of Decision Tree and Random Forest models can be compared.

Decision Tree Classifier

A Decision Tree is another supervised machine learning algorithm commonly used for classification problems. It represents the decision-making process in the form of a tree structure.

A Decision Tree consists of nodes, branches, and leaf nodes. The internal nodes represent decisions based on features, branches represent possible outcomes of those decisions, and leaf nodes represent the final predictions.

For example, a Decision Tree may first examine a particular feature such as ChestPainType. Depending on the value of that feature, the model may follow one branch or another. It may then examine another feature, such as MaxHR or Oldpeak, before eventually reaching a final classification. The algorithm builds the tree by selecting features and split points that provide useful separation between the target classes. Various criteria can be used to determine the quality of a split, including Gini impurity and entropy. Gini impurity measures how mixed the classes are within a particular node. A good split generally produces child nodes containing more clearly separated classes. The Decision Tree approach is relatively easy to understand because its decisions can be represented visually. This makes it more interpretable than some complex machine learning models. Another advantage is that Decision Trees can handle both numerical and categorical information after appropriate preprocessing. They can also capture nonlinear relationships between features and the target. However, Decision Trees can suffer from overfitting. A tree that becomes too deep may learn the training data too closely, including patterns that do not generalize well to unseen data. This can result in very high training performance but lower testing performance. To reduce overfitting, parameters such as maximum tree depth, minimum samples required for a split, and minimum samples required at a leaf can be controlled. In this project, the Decision Tree Classifier is trained using the prepared heart disease dataset. Its predictions are evaluated using the testing dataset and compared with the other selected algorithms.

Random Forest Classifier

Random Forest is an ensemble machine learning algorithm that combines multiple Decision Trees to produce a final prediction. Instead of depending on a single tree, Random Forest creates a collection of trees and combines their predictions. The basic idea behind Random Forest is that multiple trees can work together to produce a more stable and reliable prediction than a single Decision Tree. During training, different trees are created using different samples and subsets of features. Each tree independently learns patterns from the training data. When a new patient record is provided, each tree produces a prediction. For classification, the final prediction is generally determined using the majority vote of the individual trees.

For example, if most trees predict that a patient belongs to class "1", the Random Forest model will classify the patient as having heart disease.

Random Forest has several advantages. One important advantage is its ability to handle complex relationships between features and the target. It can capture nonlinear patterns that may be difficult for simpler models such as Logistic Regression to represent. Another advantage is that Random Forest generally provides better protection against overfitting compared with a single unrestricted Decision Tree. Since the final prediction is based on multiple trees, the model can be more robust to variations in individual training samples. Random Forest can also provide information about feature importance. Feature importance can help identify which input variables contributed more strongly to the model's predictions. For a heart disease dataset, this may help in understanding which available features the model considers useful for classification. However, Random Forest is more complex than Logistic Regression and a single Decision Tree. Because it contains multiple trees, it may require more computational resources and can be more difficult to interpret directly. Despite this, Random Forest is widely used for classification tasks because of its flexibility and strong performance on many types of structured datasets.

Comparison of the Selected Algorithms

The three algorithms used in this project have different characteristics.

Algorithm	Basic Approach	Main Advantage	Main Limitation
Logistic Regression	Probability-based classification	Simple and efficient	Limited for complex nonlinear relationships
Decision Tree	Tree-based decision rules	Easy to understand	Can overfit
Random Forest	Multiple decision trees	More robust and flexible	More complex and less interpretable

Logistic Regression provides a simple baseline model. Decision Tree provides a rule-based classification approach that is relatively easy to visualize and understand. Random Forest extends the tree-based approach by combining multiple trees to improve generalization.

Comparing these algorithms allows the project to determine which approach is most suitable for the selected heart disease dataset.

Importance of Using Multiple Algorithms

Using multiple algorithms is important because no single machine learning algorithm is guaranteed to provide the best performance for every dataset. Logistic Regression may perform well when the relationship between the input variables and target is relatively simple. Decision Tree may perform better when the data contains nonlinear decision patterns. Random Forest may provide stronger generalization by combining several decision trees. Therefore, comparing the three algorithms provides experimental evidence about which model performs best on the selected heart disease dataset.

Avoiding Unfair Comparison

For a meaningful comparison, all algorithms should be evaluated under similar conditions. The same dataset should be used for training and testing all models. The same train-test split should be used for each algorithm. The same target variable should be used, and the same evaluation metrics should be calculated. This ensures that differences in performance are primarily related to the algorithms themselves rather than differences in the data used for evaluation.

Role of Hyperparameters

Machine learning algorithms contain parameters that can influence their performance. Parameters selected before training are generally called hyperparameters. For example, Random Forest contains parameters such as the number of trees, maximum tree depth, and minimum samples required for splitting. Decision Tree also contains parameters such as maximum depth and splitting criteria. In a basic academic project, default parameters provided by Scikit-learn may be used initially. If required, selected hyperparameters can be adjusted to improve model performance. However, any changes made to the model parameters should be documented in the project report so that the experiment remains understandable and reproducible.

Model Evaluation After Training

Training a model alone is not enough to determine whether it is effective. After training, the models are evaluated using the testing dataset. The predictions are compared with the actual target values. Metrics such as accuracy, precision, recall, and F1-score are calculated. The confusion matrix is also generated for each model. This helps identify True Positives, True Negatives, False Positives, and False Negatives. The results of the three algorithms are then organized into a comparison table.

Expected Outcome

The purpose of applying these three algorithms is not to assume in advance which algorithm will perform best. Instead, the actual testing results obtained from the Jupyter Notebook are used to determine the best-performing algorithm. The final result may show that one algorithm achieves higher accuracy, while another may achieve better recall or precision. Therefore, all relevant evaluation metrics should be considered before selecting the final model. The best-performing model will be identified in the later section of the report after analyzing the actual experimental results.

EVALUATION METRICS

Evaluation is an important stage in any machine learning project because it helps determine how well a trained model performs on unseen data. In this heart disease prediction project, three different machine learning algorithms are trained and their predictions are evaluated using the testing dataset. The actual target values are compared with the values predicted by each model. A machine learning model should not be judged only by whether it produces predictions. Its predictions must be measured using suitable evaluation metrics. Different metrics provide different information about the quality of a classification model. In this project, the major evaluation metrics considered are Accuracy, Precision, Recall, and F1-Score. A Confusion Matrix is also used to provide a detailed understanding of the correct and incorrect predictions.

Since the "HeartDisease" target variable contains two classes, the project is treated as a binary classification problem. The two classes are represented as:

0 — No heart disease

-1 — Heart disease present

The evaluation metrics help compare the performance of Logistic Regression, Decision Tree, and Random Forest models.

Confusion Matrix Components

Before understanding the evaluation metrics, it is important to understand the four basic outcomes of a binary classification problem. These outcomes are obtained by comparing the actual class with the class predicted by the machine learning model.

The four outcomes are:

True Positive (TP): A patient who actually has heart disease is correctly predicted as having heart disease.

True Negative (TN): A patient who does not have heart disease is correctly predicted as not having heart disease.

False Positive (FP): A patient who does not have heart disease is incorrectly predicted as having heart disease.

False Negative (FN): A patient who actually has heart disease is incorrectly predicted as not having heart disease.

These four values form the foundation of most classification evaluation metrics.

The confusion matrix can be represented as:

Actual / Predicted| No Heart Disease| Heart Disease

No Heart Disease| TN| FP

Heart Disease| FN| TP

Understanding these values makes it easier to interpret the performance of each machine learning algorithm.

Accuracy

Accuracy is one of the most commonly used classification evaluation metrics. It represents the proportion of total predictions that are correctly classified by the model.

The formula for accuracy is:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

In simple terms, accuracy answers the question:

"Out of all the predictions made by the model, how many were correct?"

For example, if a model correctly classifies 90 out of 100 patients, its accuracy is 90%.

Accuracy is easy to understand and useful when the classes are reasonably balanced. A higher accuracy generally indicates that the model is making a larger proportion of correct predictions. However, accuracy alone may not always provide a complete picture of model performance. If the dataset is imbalanced, a model could achieve high accuracy while performing poorly on the minority class. Therefore, accuracy should be considered together with precision, recall, and F1-score. In this project, accuracy is calculated for all three selected machine learning algorithms and used as one of the measures for comparison.

Precision

Precision measures the correctness of the positive predictions made by the model.

The formula for precision is:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Precision answers the question:

"Out of all patients predicted as having heart disease, how many actually have heart disease?"

A high precision value means that the model produces relatively few false-positive predictions.

For example, suppose a model predicts that 50 patients have heart disease, but only 45 of them actually have heart disease. The precision would be:

$$45 / 50 = 0.90$$

Therefore, the precision would be 90%.

Precision is useful when false-positive predictions are important to consider. In the context of this project, a false positive means that the model predicts heart disease for a patient who does not actually belong to the heart disease class.

A higher precision indicates that when the model predicts the positive class, its prediction is more likely to be correct.

Recall

Recall is also known as sensitivity or True Positive Rate. It measures the ability of the model to correctly identify actual positive cases.

The formula for recall is:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Recall answers the question:

"Out of all patients who actually have heart disease, how many did the model correctly identify?"

For example, suppose 50 patients actually have heart disease and the model correctly identifies 45 of them. The recall would be:

$$45 / 50 = 0.90$$

Therefore, the recall would be 90%.

Recall is particularly important in many medical classification problems because failing to identify an actual positive case can be significant. A false negative occurs when a patient who actually has heart disease is predicted as not having heart disease.

Therefore, a model with a high recall is effective at identifying a larger proportion of patients who actually belong to the heart disease class.

However, increasing recall can sometimes result in more false-positive predictions. Therefore, recall should be analyzed together with precision and other metrics rather than considered independently.

F1-Score

F1-score is a classification metric that combines precision and recall into a single value. It is particularly useful when both precision and recall are important.

The formula for F1-score is:

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1-score is the harmonic mean of precision and recall.

Its value ranges from 0 to 1, where a value closer to 1 indicates better performance.

For example, if a model has high precision but very low recall, its F1-score will not be extremely high. Similarly, if the model has high recall but low precision, the F1-score will also be affected.

Therefore, the F1-score provides a balanced measure of performance when both types of errors need to be considered. In this project, F1-score is calculated for each machine learning algorithm and used along with accuracy, precision, and recall to compare the models.

Classification Report

A classification report provides several evaluation metrics together in a structured format. It can contain precision, recall, F1-score, and support for each target class. The classification report makes it easier to observe how the model performs for both classes rather than considering only the overall accuracy.

For the heart disease dataset, the classification report can show separate results for:

Class 0 — No heart disease

Class 1 — Heart disease

This is useful because a model may perform differently for the two classes.

The classification report also provides the average performance across the classes. These values can be considered while comparing the machine learning algorithms.

Support

Support represents the number of actual observations belonging to each class in the testing dataset.

For example, if the testing dataset contains 100 patient records and 55 belong to class 0 while 45 belong to class 1, then the support values are 55 and 45 respectively.

Support itself is not a performance metric, but it helps provide context when interpreting precision, recall, and F1-score. If one class has significantly fewer observations than another, the performance metrics should be interpreted carefully.

Importance of Recall in Heart Disease Prediction

In a healthcare-related classification problem, recall can be particularly important. The reason is that a false negative means that the model fails to identify a patient who actually belongs to the heart disease class. A high recall means that the model successfully identifies a larger proportion of actual positive cases.

For example, if 100 patients actually have heart disease and a model correctly identifies 95 of them, the recall is 95%. If the model identifies only 60 of those 100 patients, its recall is 60%.

Therefore, when comparing models in this project, recall should receive careful attention along with the other evaluation metrics. However, this does not mean that the model with the highest recall should automatically be selected. Precision, F1-score, accuracy, and confusion matrix results should also be considered.

Importance of Precision

Precision is also useful because it indicates how reliable the positive predictions are. A model with low precision produces a relatively large number of false-positive predictions. In this project, that means the model predicts heart disease for patients who actually belong to the non-heart-disease class. High precision indicates that the model's positive predictions are more reliable. Therefore, precision and recall should generally be examined together. A model that performs well on one metric but poorly on another may not necessarily be the most suitable overall model.

Comparing the Models

The evaluation metrics are calculated for each of the three machine learning algorithms. The results can be organized into a table such as:

Algorithm| Accuracy| Precision| Recall| F1-Score

Logistic Regression| Actual Result| Actual Result| Actual Result| Actual Result

Decision Tree| Actual Result| Actual Result| Actual Result| Actual Result

Random Forest| Actual Result| Actual Result| Actual Result| Actual Result

The phrase "Actual Result" in the above example should be replaced with the values obtained from the Jupyter Notebook.

For example, after running the models, the table might contain values such as 0.85, 0.88, or 0.90. These values should not be guessed or manually entered before running the actual models.

Graphical Comparison

The evaluation results can also be presented using graphs. A bar chart can be created to compare accuracy, precision, recall, and F1-score among the three algorithms. Graphical comparison makes it easier to identify differences in model performance. For example, the x-axis can contain the names of the algorithms, while the y-axis can represent the evaluation score. Separate comparisons can be made for accuracy, precision, recall, and F1-score.

The graphs can be included in the project report as evidence of the model comparison.

Confusion Matrix Visualization

In addition to numerical metrics, confusion matrices can be displayed visually using a heatmap.

The confusion matrix provides information about:

True Positives

True Negatives

False Positives

False Negatives

A confusion matrix visualization makes it easier to understand which classes the model predicts correctly and where errors occur. For each machine learning algorithm, a separate confusion matrix can be generated. These matrices can then be compared to understand which model produces fewer incorrect classifications.

Why Multiple Metrics Are Necessary

No single metric is sufficient to completely describe the performance of a classification model. Accuracy gives an overall measure of correctness, but it does not distinguish between different types of errors.

Precision focuses on the correctness of positive predictions.

Recall focuses on the ability to identify actual positive cases.

F1-score provides a balance between precision and recall.

The confusion matrix provides the underlying counts of correct and incorrect predictions.

Therefore, using multiple evaluation measures gives a more complete understanding of the strengths and weaknesses of each model.

Selection of the Best Model Using Evaluation Metrics

After calculating all the metrics, the algorithms are compared to identify the best-performing model.

The best model should not be selected based solely on accuracy. Instead, all relevant metrics should be considered.

For example, if one model has the highest accuracy but significantly lower recall, while another model has slightly lower accuracy but much better recall and F1-score, the second model may be worth considering depending on the objective of the application. For this heart disease prediction project, the ability to identify actual heart disease cases is important. Therefore, recall should receive particular attention while also considering precision, F1-score, accuracy, and confusion matrix results. The final best-performing algorithm will be identified after the actual testing results have been obtained.

Interpretation of Metric Values

Generally, higher values of accuracy, precision, recall, and F1-score indicate better classification performance.

For example:

Accuracy closer to 1: More overall predictions are correct.

Precision closer to 1: Positive predictions are more reliable.

Recall closer to 1: More actual positive cases are correctly identified.

F1-score closer to 1: Better balance between precision and recall.

However, the interpretation should always consider the dataset and the purpose of the classification problem. A model should not be considered good simply because one metric has a high value. The complete set of evaluation results should be examined.

Summary

Evaluation metrics provide an objective method for measuring the performance of the machine learning algorithms used in this project. Since the heart disease prediction problem is a binary classification task, the models are evaluated by comparing their predicted classes with the actual classes in the testing dataset. Accuracy measures the overall percentage of correct predictions.

Precision measures the correctness of positive predictions. Recall measures the ability to identify actual heart disease cases, while F1-score provides a balance between precision and recall. The confusion matrix provides additional information by showing True Positives, True Negatives, False Positives, and False Negatives. These values help explain the types of errors made by each algorithm.

In this project, Logistic Regression, Decision Tree, and Random Forest are evaluated using the same testing dataset. Their accuracy, precision, recall, and F1-score values are recorded and compared. Confusion matrices are also generated to provide a detailed analysis of their predictions.

The final model selection is based on an overall analysis of these evaluation results. Particular attention is given to recall because correctly identifying patients who actually have heart disease is an important consideration in this healthcare-related classification problem.

Thus, evaluation metrics play a crucial role in determining the effectiveness of the machine learning models and provide the evidence required to compare the algorithms and identify the best-performing model.

Results and Comparison

The results and comparison section presents the performance of the machine learning algorithms applied to the heart disease dataset. In this project, three machine learning algorithms were selected and implemented: Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. The main purpose of comparing these algorithms is to determine which model provides better performance for predicting the presence or absence of heart disease. After preprocessing the dataset, the data was divided into training and testing sets. The training data was used to train the three selected machine learning models, while the testing data was used to evaluate their performance on previously unseen data. This process helps determine whether the models can generalize their learning to new patient records. The performance of each algorithm was evaluated using several important classification metrics, including Accuracy, Precision, Recall, and F1-Score. Using multiple evaluation metrics provides a more complete understanding of model performance rather than depending only on accuracy. This is particularly important in heart disease prediction because incorrectly classifying a patient may have different consequences depending on whether the patient actually has heart disease or not.

Performance Comparison

The results obtained from the Jupyter Notebook are organized in a comparison table. The actual values should be entered from the outputs generated after running the machine learning models. This table provides a simple way to compare the three algorithms. Generally, a higher value for accuracy indicates that the model correctly classified a larger proportion of the test records. Precision indicates how accurately the model identifies positive predictions, while recall measures how effectively the model identifies actual positive cases. The F1-score provides a balance between precision and recall.

Logistic Regression Results

Logistic Regression was used as one of the classification algorithms because it is a commonly used and relatively simple method for binary classification problems. In this project, the model attempts to classify each record into one of two classes: no heart disease or heart disease. After training the Logistic Regression model using the training dataset, predictions were generated for the testing dataset. The predictions were then evaluated using accuracy, precision, recall, and F1-score. The actual performance values obtained from the Jupyter Notebook should be entered into the results table. These values can then be compared with the results of the Decision Tree and Random Forest models. Logistic Regression is useful as a baseline model because it provides a straightforward method for determining whether the selected features contain enough information to classify the target variable. Its performance also provides a reference point for evaluating whether the more complex Decision Tree and Random Forest models provide better results.

Decision Tree Classifier Results

The Decision Tree Classifier was the second algorithm applied to the dataset. Unlike Logistic Regression, a Decision Tree makes predictions by creating a series of decision rules based on the input features. The tree contains different nodes and branches that lead to a final prediction. After training the Decision Tree model, predictions were made using the test dataset. The predictions were evaluated using the same metrics used for Logistic Regression. Using the same evaluation metrics makes the comparison between the algorithms fair and meaningful. The accuracy, precision, recall, and F1-score obtained from the Decision Tree model should be recorded in the comparison table. The confusion matrix can also be used to examine the number of correctly and incorrectly classified instances. One advantage of the Decision Tree is its interpretability. The decision-making process can be easier to understand because it follows a sequence of conditions. However, a Decision Tree may become too complex and overfit the training data if it is not appropriately controlled.

Random Forest Classifier Results

The Random Forest Classifier was used as the third machine learning algorithm. Random Forest is an ensemble learning technique that combines multiple decision trees to make predictions. Instead of depending on a single tree, the model considers the predictions of several trees and produces a final classification. The Random Forest model was trained using the training dataset and evaluated using the testing dataset. Its performance was measured using accuracy, precision, recall, and F1-score. The actual values obtained from the Jupyter Notebook should be entered into the results table. These results can then be compared with Logistic Regression and Decision Tree Classifier. Random Forest can often provide strong classification performance because it combines multiple decision trees and reduces the dependence on a single decision-making structure. However, the actual performance in this project must be determined from the experimental results rather than assumed in advance.

Comparison of Algorithms

The three algorithms are compared using the same testing dataset and evaluation metrics. This provides a fair basis for determining which algorithm performs better. If one model has the highest accuracy, it means that it correctly classified the largest proportion of test records. However, accuracy alone should not be used to select the final model. Precision, recall, and F1-score should also be considered. For a heart disease classification problem, recall is particularly important because it indicates how many of the actual heart disease cases were correctly identified by the model. A model with higher recall may be preferable when reducing false-negative predictions is an important consideration. Precision is also useful because it indicates how many of the records predicted as having heart disease were actually positive cases. The F1-score provides a combined measure of precision and recall and can be useful when both measures are important.

Confusion Matrix Comparison

In addition to numerical evaluation metrics, confusion matrices can be used to compare the three models. A confusion matrix contains four important values: True Positive, True Negative, False Positive, and False Negative. A True Positive (TP) represents a case that actually has heart disease and was correctly predicted as having heart disease. A True Negative (TN) represents a case without heart disease that was correctly predicted as negative. A False Positive (FP) occurs when the model predicts heart disease for a person who does not have it. A False Negative (FN) occurs when the model predicts no heart disease for a person who actually has heart disease. The confusion matrices generated in the Jupyter Notebook can therefore be included in the report as screenshots or figures. Comparing these matrices helps identify the types of errors made by each algorithm.

Graphical Comparison

The performance metrics can also be presented using a bar graph. The graph can display the accuracy, precision, recall, and F1-score of Logistic Regression, Decision Tree, and Random Forest. A graphical representation makes it easier to identify differences between the models. For example, if one algorithm consistently produces higher values across several metrics, it may be considered stronger than the other algorithms. The graph should be generated using the actual results obtained from the Jupyter Notebook. The graph can then be inserted into the report under the Results and Comparison section.

Identification of the Best Model

The best-performing algorithm should be selected based on the actual experimental results. The algorithm with the strongest overall performance across the important evaluation metrics can be considered the best model for this particular dataset and experiment.

Confusion Matrix Analysis

A confusion matrix is an important evaluation technique used to understand the classification performance of a machine learning model. In this heart disease prediction project, the confusion matrix is used to analyze how accurately the selected machine learning algorithms classify patients into two categories: Heart Disease Present and Heart Disease Absent. Although accuracy provides an overall measurement of model performance, it does not explain the types of mistakes made by the model. A confusion matrix provides more detailed information by showing the number of correct and incorrect predictions for each class. Therefore, it is useful for understanding the strengths and weaknesses of Logistic Regression, Decision Tree Classifier, and Random Forest Classifier.

Structure of a Confusion Matrix

Since the target variable "HeartDisease" contains two classes, the problem is a binary classification problem. The confusion matrix consists of four main components:

Actual Class| Predicted Negative| Predicted Positive

Negative| True Negative (TN)| False Positive (FP)

Positive| False Negative (FN)| True Positive (TP)

The four components are explained below.

True Negative (TN):

A True Negative occurs when the patient does not have heart disease and the machine learning model correctly predicts that the patient does not have heart disease.

True Positive (TP):

A True Positive occurs when the patient has heart disease and the model correctly predicts the presence of heart disease.

False Positive (FP):

A False Positive occurs when the patient does not have heart disease, but the model incorrectly predicts that the patient has heart disease.

False Negative (FN):

A False Negative occurs when the patient actually has heart disease, but the model incorrectly predicts that the patient does not have heart disease.

Importance of False Negatives

False Negative predictions are particularly important to consider in a heart disease prediction problem. A false negative means that a patient who actually has heart disease is incorrectly classified as not having heart disease. From a machine learning evaluation perspective, reducing false negatives can improve the recall of the model. Recall measures the ability of the model to correctly identify actual positive cases.

The formula for recall is:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Therefore, if the number of false negatives decreases while the number of true positives remains high, the recall value generally improves.

However, this project is an academic machine learning experiment and not a medical diagnostic system. The predictions generated by the models should not be used to make real medical decisions.

Confusion Matrix for Logistic Regression

The first model used in this project is Logistic Regression. After training the model using the training dataset, predictions are generated for the testing dataset. These predictions can be compared with the actual target values to create the confusion matrix. The confusion matrix for Logistic Regression should be generated in the Jupyter Notebook and inserted into the report.

Insert the Logistic Regression confusion matrix screenshot generated from Jupyter Notebook here. The matrix can be examined by observing the True Positive, True Negative, False Positive, and False Negative values. A larger number of correct predictions indicates better classification performance, while a smaller number of incorrect predictions indicates fewer classification errors.

Confusion Matrix for Decision Tree Classifier

The second model used in this project is the Decision Tree Classifier. After training the Decision Tree using the training dataset, predictions are generated for the test dataset.

The actual and predicted values are then used to create the confusion matrix.

Insert the Decision Tree confusion matrix screenshot generated from Jupyter Notebook here.

The confusion matrix helps determine how many records were classified correctly and how many were incorrectly classified. It also shows whether the model produces more false positives or false negatives.

Confusion Matrix for Random Forest Classifier

The third model used in this project is the Random Forest Classifier. Random Forest uses multiple decision trees to make the final classification. After making predictions on the test dataset, the actual and predicted values are used to generate the confusion matrix. Insert the Random Forest confusion matrix screenshot generated from Jupyter Notebook here. The confusion matrix can be examined to determine the number of correct and incorrect predictions. It can also be compared with the confusion matrices of Logistic Regression and Decision Tree.

Comparison of Confusion Matrices

The confusion matrices of the three algorithms provide a detailed comparison of their classification behavior.

Model	True Positive	True Negative	False Positive	False Negative
-------	---------------	---------------	----------------	----------------

Logistic Regression	[Value]	[Value]	[Value]	[Value]
---------------------	---------	---------	---------	---------

Decision Tree	[Value]	[Value]	[Value]	[Value]
---------------	---------	---------	---------	---------

Random Forest	[Value]	[Value]	[Value]	[Value]
---------------	---------	---------	---------	---------

The values in this table should be entered directly from the confusion matrix outputs generated in the Jupyter Notebook. A model with more True Positives and True Negatives generally makes more correct predictions. At the same time, fewer False Positives and False Negatives indicate fewer classification errors. However, the importance of each type of error depends on the objective of the classification problem.

Relationship Between Confusion Matrix and Evaluation Metrics

The values in a confusion matrix are directly related to the evaluation metrics used in this project.

Accuracy is calculated as:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Precision is calculated as:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall is calculated as:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-score is calculated using precision and recall:

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Therefore, the confusion matrix provides the basic values required to calculate and understand several important evaluation metrics.

Visualization of Confusion Matrix

For better understanding, confusion matrices can be displayed as heatmaps using Python visualization libraries. A heatmap represents the values using different shades, making it easier to identify where the model has made correct and incorrect predictions.

The confusion matrix generated in Jupyter Notebook should be captured as a screenshot and added to the report. Each model should have its own confusion matrix figure.

The figures should be clearly labelled, for example:

Interpretation of the Results

The confusion matrix analysis provides more information than accuracy alone. Two models can have similar accuracy values but different numbers of false positives and false negatives. Therefore, examining the confusion matrix helps identify the type of errors made by each algorithm. For example, if one model has a high number of False Negatives, its ability to identify positive heart disease cases may be weaker. If another model has fewer False Negatives and a higher recall value, it may be more suitable according to the objective of identifying positive cases. Similarly, a model with many False Positives may produce more incorrect positive predictions. This can result in lower precision. Therefore, the confusion matrices should be considered together with accuracy, precision, recall, and F1-score when comparing the three machine learning algorithms.

Summary

The confusion matrix is an important tool for analyzing the performance of classification models. In this project, it is used to evaluate Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. The matrix provides four important values: True Positive, True Negative, False Positive, and False Negative. These values help explain the correct and incorrect predictions made by each model.

The confusion matrix analysis, along with accuracy, precision, recall, and F1-score, provides a complete understanding of the performance of the selected algorithms. The actual confusion matrix values obtained from the Jupyter Notebook should be included in the report.

Finally, the confusion matrices can be used along with the other evaluation results to identify the most suitable performing model for this particular dataset and experimental setup.

Best Performing Algorithm

The main purpose of applying multiple machine learning algorithms in this project is to identify the algorithm that provides the most suitable performance for the selected heart disease dataset. Three classification algorithms, namely Logistic Regression, Decision Tree Classifier, and Random Forest Classifier, were implemented and evaluated. The models were trained using the training dataset and tested using the testing dataset. The performance of the three algorithms was evaluated using different classification metrics such as Accuracy, Precision, Recall, and F1-Score. Confusion matrices were also generated to understand the correct and incorrect predictions made by each model. By considering these results together, the best-performing algorithm can be identified.

Criteria for Selecting the Best Algorithm

The selection of the best algorithm should not be based on only one evaluation metric. Different metrics provide different information about model performance. Accuracy indicates the overall percentage of correctly classified records. A higher accuracy means that the model correctly predicts a larger proportion of the test dataset. Precision indicates how many of the records predicted as positive are actually positive. A higher precision means that the model produces fewer false-positive predictions. Recall indicates how effectively the model identifies actual positive cases. In this project, recall is particularly important

because the positive class represents the presence of heart disease. F1-Score combines precision and recall into a single measure. A higher F1-score indicates a better balance between precision and recall. Therefore, all these metrics should be considered before selecting the final model.

Comparison of the Three Algorithms

The performance results obtained from the Jupyter Notebook should be summarized in the following table.

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	[Value]	[Value]	[Value]	[Value]
Decision Tree Classifier	[Value]	[Value]	[Value]	[Value]
Random Forest Classifier	[Value]	[Value]	[Value]	[Value]

The values should be copied directly from the model evaluation outputs generated in the Jupyter Notebook. This ensures that the report accurately represents the experimental results.

Logistic Regression as a Candidate Model

Logistic Regression is a commonly used classification algorithm and provides a useful baseline for this project. It is relatively simple and can perform well when the relationship between the input features and target variable can be represented effectively by the model. Its performance can be evaluated by examining its accuracy, precision, recall, and F1-score. If Logistic Regression produces consistently strong values across the important metrics, it may be considered a suitable model for this dataset. However, its final position in the comparison should be determined only after comparing its actual results with the Decision Tree and Random Forest models.

Decision Tree as a Candidate Model

The Decision Tree Classifier makes predictions using a sequence of decision rules. It is easy to understand and can capture non-linear relationships between input features.

The Decision Tree results should be compared with the results of Logistic Regression and Random Forest. If the Decision Tree produces higher evaluation scores and fewer classification errors, it may be considered the better-performing model. However, a Decision Tree can sometimes become too complex and may overfit the training data. Therefore, its performance on the test dataset is more important than its performance on the training dataset.

Random Forest as a Candidate Model

Random Forest is an ensemble learning algorithm that combines multiple decision trees. Each tree contributes to the final prediction, which can make the model more robust than relying on a single decision tree. The Random Forest results should be compared with Logistic Regression and Decision Tree using the same test dataset and evaluation metrics. If it produces strong accuracy, precision, recall, and F1-score while maintaining a good confusion matrix, it may be selected as the best-performing algorithm. However, the model should not automatically be declared the best simply because Random Forest is generally considered a powerful algorithm. The final decision must be based on the actual results obtained from this particular dataset.

Confusion Matrix and Best Model Selection

The confusion matrices of the three models provide additional information for selecting the best-performing algorithm. A model with a high number of True Positives and True Negatives and a low number of False Positives and False Negatives generally demonstrates better classification performance. For this heart disease prediction project, False Negatives are particularly important to examine because they represent cases where the model predicts that heart disease is absent when the actual class indicates

heart disease. Therefore, the model's recall and False Negative count should be carefully considered along with the other performance metrics.

Overall Model Selection

After comparing all the results, the algorithm that provides the most suitable overall performance can be selected as the best-performing model.

The final statement in the report should be written using the actual results. For example:

“Based on the experimental results, [MODEL NAME] achieved the best overall performance among the three algorithms. It obtained an accuracy of [VALUE], precision of [VALUE], recall of [VALUE], and F1-score of [VALUE]. Therefore, [MODEL NAME] was identified as the best-performing algorithm for the selected heart disease dataset.”

The values in this statement must be replaced with the actual values obtained from the Jupyter Notebook.

Importance of the Best Model

Identifying the best-performing algorithm is an important part of the machine learning workflow. Different algorithms may perform differently on the same dataset because they use different learning approaches. The best model identified in this project can demonstrate which algorithm was more effective under the specific preprocessing steps, train-test split, features, and experimental conditions used in the study. It is also important to understand that the best model for one dataset is not necessarily the best model for every heart disease dataset. Model performance depends on factors such as dataset size, feature quality, preprocessing methods, class distribution, and model parameters.

Academic Interpretation

The selection of the best-performing algorithm in this project is based on experimental machine learning evaluation rather than medical diagnosis. The project demonstrates how classification algorithms can be trained and compared using a structured dataset. The results show how different algorithms can produce different prediction performances when applied to the same data. By using multiple evaluation metrics and confusion matrices, the comparison becomes more reliable than using accuracy alone. This process also provides practical understanding of the machine learning workflow, including dataset preparation, feature processing, model training, prediction, evaluation, comparison, and selection of a suitable model.

Summary

In this project, Logistic Regression, Decision Tree Classifier, and Random Forest Classifier were applied to the heart disease dataset. Their performance was evaluated using accuracy, precision, recall, F1-score, and confusion matrices. The best-performing algorithm should be selected by comparing the actual experimental results of all three models. Particular attention should be given to recall and false-negative predictions because the positive class represents the presence of heart disease. The final model should therefore be selected based on the overall evaluation results rather than assumptions about which algorithm is generally better. The actual values from the Jupyter Notebook will provide the evidence required to identify the best-performing algorithm for this project.

Sample Code and Outputs

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
df = pd.read_csv("../dataset/heart.csv")
df.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
```

```
df.shape
```

```
(918, 12)
```

```
df.info()
df.describe()
df.columns
df.isnull().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   Age                 918 non-null   int64
 1   Sex                 918 non-null   str
 2   ChestPainType       918 non-null   str
 3   RestingBP           918 non-null   int64
 4   Cholesterol          918 non-null   int64
 5   FastingBS           918 non-null   int64
 6   RestingECG          918 non-null   str
 7   MaxHR               918 non-null   int64
 8   ExerciseAngina       918 non-null   str
 9   Oldpeak             918 non-null   float64
10   ST_Slope            918 non-null   str
11   HeartDisease         918 non-null   int64
dtypes: float64(1), int64(6), str(5)
memory usage: 86.2 KB

Age                0
Sex                0
ChestPainType      0
RestingBP          0
Cholesterol         0
FastingBS          0
RestingECG         0
MaxHR              0
ExerciseAngina     0
Oldpeak            0
ST_Slope           0
HeartDisease       0
dtype: int64
```

```
df['HeartDisease'].value_counts()
```

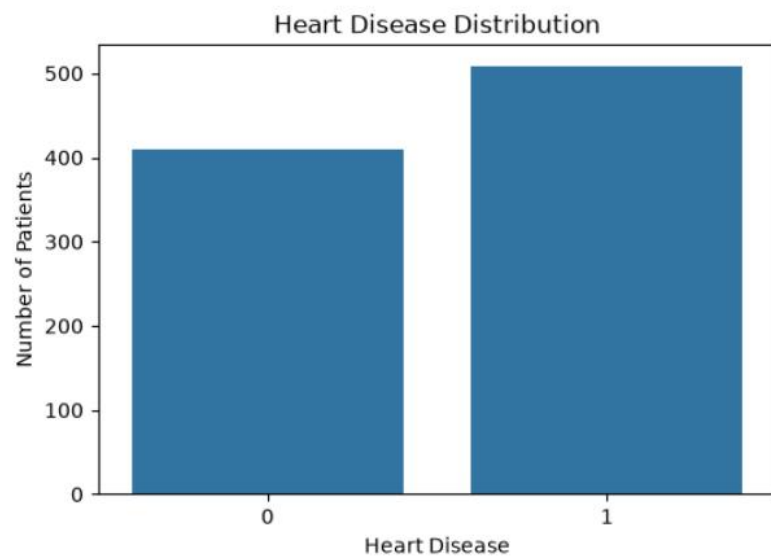
```
HeartDisease
1    508
0    410
Name: count, dtype: int64
```

```
plt.figure(figsize=(6,4))

sns.countplot(x='HeartDisease', data=df)

plt.title('Heart Disease Distribution')
plt.xlabel('Heart Disease')
plt.ylabel('Number of Patients')

plt.show()
```



```
data.select_dtypes(include='str').columns
```

```
Index(['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope'], dtype='str')
```

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

for column in data.select_dtypes(include='str').columns:
    data[column] = label_encoder.fit_transform(data[column])

data.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	1	1	140	289	0	1	172	0	0.0	2	0
1	49	0	2	160	180	0	1	156	0	1.0	1	1
2	37	1	1	130	283	0	2	98	0	0.0	2	0
3	48	0	0	138	214	0	1	108	1	1.5	1	1
4	54	1	2	150	195	0	1	122	0	0.0	2	0

```
data.isnull().sum()
```

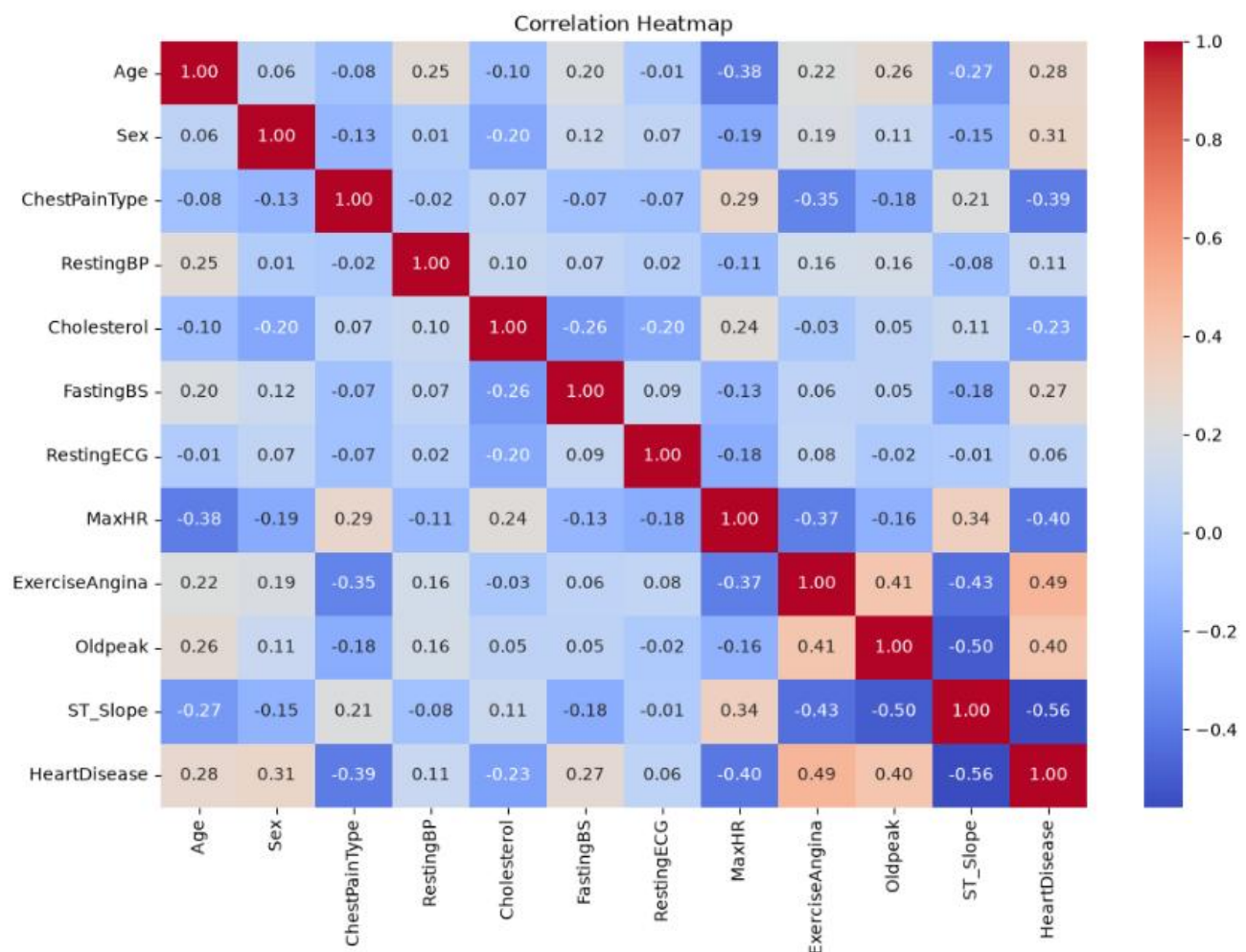
```
Age          0
Sex          0
ChestPainType  0
RestingBP    0
Cholesterol  0
FastingBS    0
RestingECG   0
MaxHR        0
ExerciseAngina  0
Oldpeak      0
ST_Slope     0
HeartDisease  0
dtype: int64
```

```
plt.figure(figsize=(12,8))

sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.2f')

plt.title('Correlation Heatmap')

plt.show()
```



```

X = data.drop('HeartDisease', axis=1)
y = data['HeartDisease']
print("Features:")
print(X.columns)

print("\nTarget:")
print(y.name)

```

```

Features:
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope'],
      dtype='str')

```

```

Target:
HeartDisease

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

```

```

Training data: (734, 11)
Testing data: (184, 11)

```

```

scaler = StandardScaler()

```

```

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

logistic_model = LogisticRegression(max_iter=1000)

```

```

logistic_model.fit(X_train_scaled, y_train)

```

```

y_pred_lr = logistic_model.predict(X_test_scaled)

```

```

accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

```

```

print("Logistic Regression")
print("Accuracy :", accuracy_lr)
print("Precision:", precision_lr)
print("Recall   :", recall_lr)
print("F1 Score :", f1_lr)

```

```

Logistic Regression
Accuracy : 0.8695652173913043
Precision: 0.8482142857142857
Recall   : 0.9313725490196079
F1 Score : 0.8878504672897196

```

```

cm_lr = confusion_matrix(y_test, y_pred_lr)

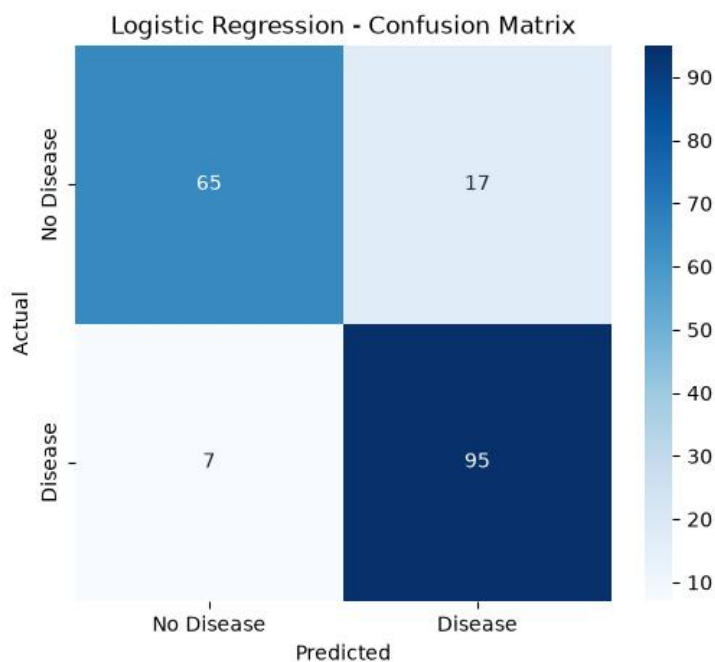
plt.figure(figsize=(6,5))

sns.heatmap(
    cm_lr,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['No Disease', 'Disease'],
    yticklabels=['No Disease', 'Disease']
)

plt.title('Logistic Regression - Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.show()

```



```

decision_tree = DecisionTreeClassifier(
    random_state=42
)

decision_tree.fit(X_train, y_train)

y_pred_dt = decision_tree.predict(X_test)

accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

print("Decision Tree Classifier")
print("Accuracy :", accuracy_dt)
print("Precision:", precision_dt)
print("Recall   :", recall_dt)
print("F1 Score :", f1_dt)

Decision Tree Classifier
Accuracy : 0.7880434782608695
Precision: 0.8118811881188119
Recall   : 0.803921568627451
F1 Score : 0.8078817733990148

```

```

cm_dt = confusion_matrix(y_test, y_pred_dt)

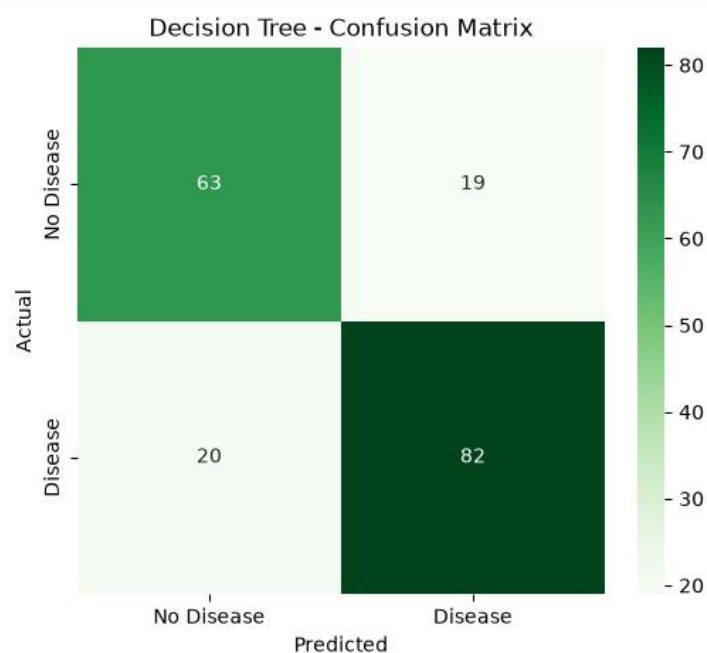
plt.figure(figsize=(6,5))

sns.heatmap(
    cm_dt,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['No Disease', 'Disease'],
    yticklabels=['No Disease', 'Disease']
)

plt.title('Decision Tree - Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.show()

```



```

random_forest = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

random_forest.fit(X_train, y_train)

y_pred_rf = random_forest.predict(X_test)

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

print("Random Forest Classifier")
print("Accuracy :", accuracy_rf)
print("Precision:", precision_rf)
print("Recall   :", recall_rf)
print("F1 Score :", f1_rf)

Random Forest Classifier
Accuracy : 0.875
Precision: 0.8761904761904762
Recall   : 0.9019607843137255
F1 Score : 0.8888888888888888

```

```

cm_rf = confusion_matrix(y_test, y_pred_rf)

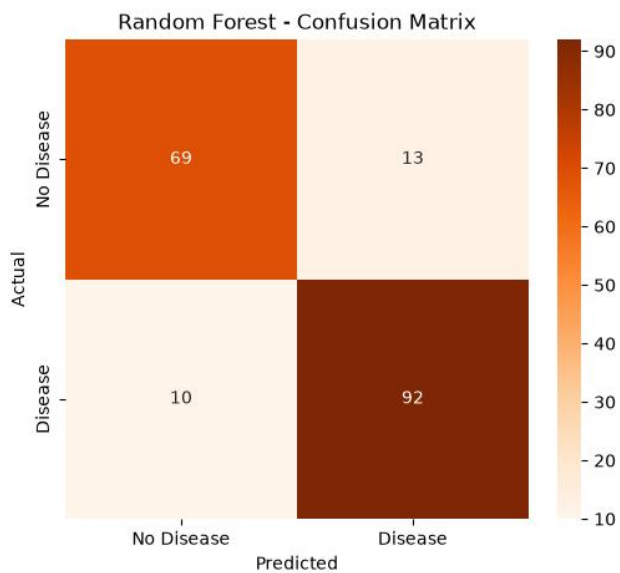
plt.figure(figsize=(6,5))

sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Oranges',
    xticklabels=['No Disease', 'Disease'],
    yticklabels=['No Disease', 'Disease']
)

plt.title('Random Forest - Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.show()

```



```

results = pd.DataFrame({
    'Algorithm': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest'
    ],
    'Accuracy': [
        accuracy_lr,
        accuracy_dt,
        accuracy_rf
    ],
    'Precision': [
        precision_lr,
        precision_dt,
        precision_rf
    ],
    'Recall': [
        recall_lr,
        recall_dt,
        recall_rf
    ],
    'F1 Score': [
        f1_lr,
        f1_dt,
        f1_rf
    ]
})

```

results

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.869565	0.848214	0.931373	0.887850
1	Decision Tree	0.788043	0.811881	0.803922	0.807882
2	Random Forest	0.875000	0.876190	0.901961	0.888889

```

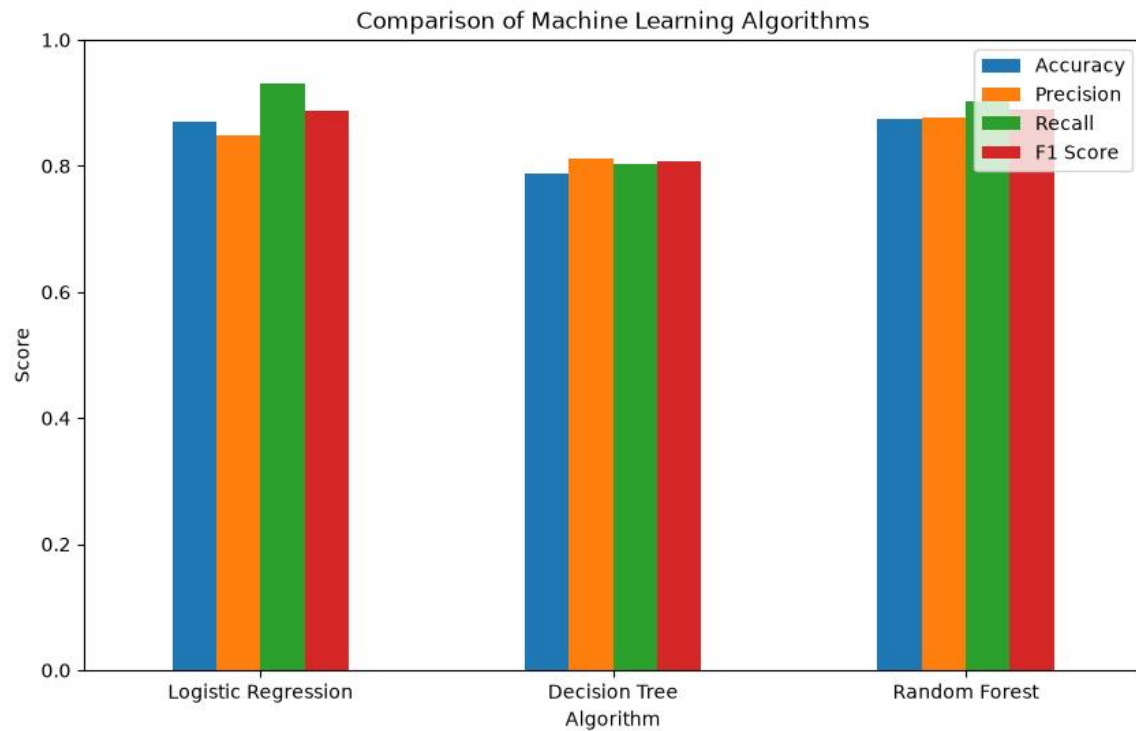
results.set_index('Algorithm').plot(
    kind='bar',
    figsize=(10,6)
)

plt.title('Comparison of Machine Learning Algorithms')
plt.ylabel('Score')
plt.ylim(0, 1)

plt.xticks(rotation=0)
plt.legend()

plt.show()

```



```

best_algorithm = results.loc[
    results['F1 Score'].idxmax()
]

print("Best Performing Algorithm:")
print(best_algorithm)

```

```

Best Performing Algorithm:
Algorithm    Random Forest
Accuracy                0.875
Precision            0.87619
Recall              0.901961
F1 Score            0.888889
Name: 2, dtype: object

```

```

best_accuracy = results.loc[
    results['Accuracy'].idxmax()
]

print("Best Algorithm Based on Accuracy:")
print(best_accuracy)

```

```

Best Algorithm Based on Accuracy:
Algorithm    Random Forest
Accuracy                0.875
Precision            0.87619
Recall              0.901961
F1 Score            0.888889
Name: 2, dtype: object

```

Conclusion

This project presented the application of machine learning techniques for predicting the presence or absence of heart disease using a structured heart disease dataset. The main objective of the project was to understand how machine learning algorithms can be applied to a classification problem and to compare the performance of different algorithms using suitable evaluation metrics. The selected dataset contained different patient-related attributes such as age, sex, chest pain type, resting blood pressure, cholesterol level, fasting blood sugar, resting electrocardiographic results, maximum heart rate, exercise-induced angina, ST depression, and ST slope. The target variable, "HeartDisease", was used to classify the records into two categories: absence of heart disease and presence of heart disease. The project followed a systematic machine learning workflow. Initially, the dataset was loaded into the Python environment using the Pandas library. The structure and characteristics of the dataset were examined to understand the available features and target variable. Dataset exploration was performed to identify missing values, duplicate records, data types, and other possible issues that could affect the model-building process. Data preprocessing was then performed to prepare the dataset for machine learning. Categorical features were converted into numerical representations so that they could be processed by machine learning algorithms. Numerical features were examined and appropriate preprocessing techniques were applied where required. The input features were separated from the target variable, with the patient attributes being used as input variables and "HeartDisease" being used as the target variable. The dataset was divided into training and testing sets. The training data was used to teach the machine learning algorithms, while the testing data was used to evaluate their performance on previously unseen records. This separation is important because evaluating a model using the same data on which it was trained may provide an unrealistic estimate of its ability to generalize to new data. Three machine learning algorithms were selected for this project: Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. Logistic Regression was used as a basic classification approach and provided a useful baseline for comparison. Decision Tree Classifier was selected because it can represent decision-making processes using a tree-like structure and can capture non-linear relationships in the data. Random Forest Classifier was selected as an ensemble learning method that combines multiple decision trees to produce a final prediction. After training the three models, predictions were generated using the test dataset. The performance of the models was evaluated using several classification metrics, including accuracy, precision, recall, and F1-score. These metrics provide different perspectives on model performance. Accuracy measures the proportion of correctly classified records among all test records. Precision measures how many of the records predicted as positive were actually positive. Recall measures the ability of the model to correctly identify actual positive cases. F1-score provides a balance between precision and recall. Using multiple metrics makes the comparison more meaningful than depending only on accuracy. Confusion matrices were also used to analyze the predictions produced by the three algorithms. A confusion matrix provides information about True Positives, True Negatives, False Positives, and False Negatives. This analysis helps identify the types of classification errors made by each model. In particular, False Negatives are important to examine in a heart disease classification problem because they represent positive cases that were incorrectly predicted as negative. The results obtained from the three models should be compared using the actual values generated in the Jupyter Notebook. The comparison table can include accuracy, precision, recall, and F1-score for Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. The final result of the comparison should be stated based on the actual experimental values. The best-performing algorithm should not be selected simply because one algorithm is generally considered more powerful. Instead, the model should be selected according to its performance on the selected dataset and under the specific preprocessing and experimental conditions used in this project. The project also demonstrates the importance of choosing suitable evaluation metrics for classification problems. A model with high accuracy may not necessarily be the best model if it produces a large number of false negatives or performs poorly in recall. Therefore, examining several evaluation metrics and the confusion matrix provides a more complete understanding of model behavior.

References

The following references were used to understand the heart disease dataset, data preprocessing techniques, machine learning algorithms, and evaluation methods used in this project.

1. Kaggle. Heart Disease Dataset. Kaggle Datasets. Available at: <https://www.kaggle.com/>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, pp. 2825–2830.
3. Scikit-learn Developers. Scikit-learn User Guide: Logistic Regression. Scikit-learn Documentation.
4. Scikit-learn Developers. Scikit-learn User Guide: Decision Tree Classifier. Scikit-learn Documentation.
5. Scikit-learn Developers. Scikit-learn User Guide: Random Forest Classifier. Scikit-learn Documentation.
6. Scikit-learn Developers. Scikit-learn User Guide: Model Evaluation and Classification Metrics. Scikit-learn Documentation.
7. McKinney, W. (2010). *Python for Data Analysis*. O'Reilly Media.
8. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
9. Hunter, J. D. (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, 9(3), pp. 90–95.
10. Waskom, M. L. (2021). “Seaborn: Statistical Data Visualization.” *Journal of Open Source Software*, 6(60), 3021.
11. Python Software Foundation. Python Documentation. Python Programming Language Documentation.
12. Jupyter Project. Jupyter Documentation. Documentation for Jupyter Notebook and related interactive computing tools.
13. Han, J., Kamber, M., and Pei, J. (2011). *Data Mining: Concepts and Techniques*. Morgan Kaufmann.
14. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
15. James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R*. Springer.

These references provided supporting information for dataset handling, data preprocessing, classification algorithms, model evaluation, data visualization, and implementation using Python-based machine learning tools. The official documentation and textbooks were particularly useful for understanding the working principles of Logistic Regression, Decision Tree Classifier, Random Forest Classifier, and classification evaluation metrics.